

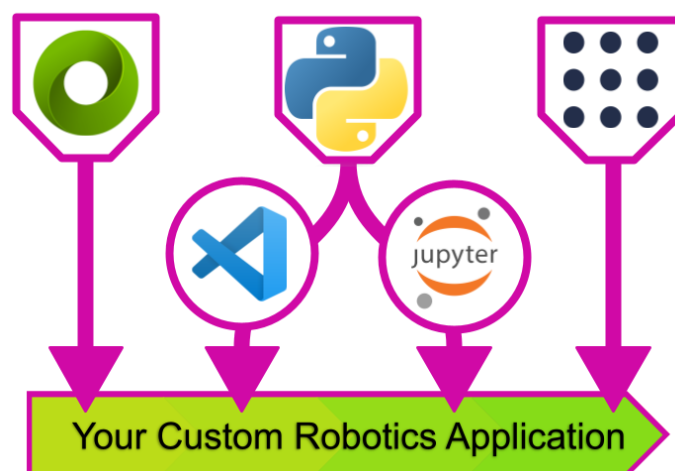


Python+VS Code 기반 Isaac Sim 개발 환경

Isaac Sim Workflows (Isaac Sim 4.0.0 + Windows)

[REF] <https://docs.omniverse.nvidia.com/isaacsim/latest/index.html>

[REF] https://docs.omniverse.nvidia.com/isaacsim/latest/introductory_tutorials/tutorial_intro_workflows.html#



- Isaac Sim은 C++과 Python을 기반으로 구축됨.
- VS Code와 Jupyter Notebooks용 Extension을 통한 Python 개발 지원.
- Synchronous 작동과 ROS 및 ROS2를 통해 하드웨어와 연동: Sim-to-Real transfer와 Digital twins을 구현함.

GUI

- Isaac Sim의 GUI 인터페이스 기능은 World-building Application인 Omniverse USD Composer에서 사용되는 기능과 동일함.
- USD Compose은 Scene을 Assemble, Illuminate, Simulate, Render하는 과정을 쉽게 만들어 크고 작은 Scene 모두에 이상적인 환경 제공.
- 주요 기능: 가상 세계 채우기 및 시뮬레이션을 위한 시각적이고 직관적인 전문 도구
- 권장 사용법: 월드 구축, 로봇 조립, 센서 부착 및 ROS 브리지 초기화

Extensions

- **Extensions**는 Omniverse Kit 기반 Application의 핵심 구성 요소임.
- **Extensions Manager**에 설치하기만 하면 다양한 Omniverse Application에서 사용할 수 있음.
- **주요 기능:** 스테이지와의 상호작용을 위해 비동기적으로 실행, 변경 사항을 즉시 반영하는 hot reloading, 실시간 시뮬레이션을 위한 적응형 물리 단계.
- **권장 사용법:** 대화형 GUI, 사용자 지정 애플리케이션 모듈 및 실시간 민감 애플리케이션 구축

Standalone Python

- **Standalone Python**은 Python Script를 사용하여 Isaac Sim을 실행.
- Script 내에서 Rendering과 Physics를 수동으로 단계별로 실행.
- 명령이 완료된 후에만 다음 단계로 진행되도록 함.
- **주요 기능:** 물리 및 렌더링 단계의 타이밍을 제어하고 headless 모드에서 실행할 수 있음.
- **권장 사용법:** 강화 학습, 체계적인 월드 생성 및 수정을 위한 대규모 학습.

실습 - Hello World

[REF] https://docs.omniverse.nvidia.com/isaacsim/latest/core_api_tutorials/tutorial_core_hello_world.html#isaac-sim-app-tutorial-core-hello-world

Extension Workflow와 Standalone Workflow에서 Python 실행하기

Isaac Examples

https://prod-files-secure.s3.us-west-2.amazonaws.com/5cf11f82-c56d-4cc4-b334-8b9699104c71/e82a651a-756d-4a10-9f8e-d3951d4badd8/Getting_Started.mp4

1. 상단 메뉴 바에서 **Isaac Examples > Hello World**를 클릭
2. **Open Source Code** 버튼을 클릭하여 **Visual Studio Code**에서 편집할 소스 코드를 시작
3. Open Containing Folder 버튼을 클릭하여 Example 파일이 들어있는 폴더를 오픈



`hello_world.py` 는 Application의 Logic이 추가되는 곳이고, Application의 UI 요소는 `hello_world_extension.py` Script에 추가되어 Logic에 연결됩니다.

4. **LOAD** 버튼을 클릭하여 World를 로드
5. **File > New From Stage Template > Empty**를 클릭하여 새 Stage를 만들고, 현재 Stage를 저장할지 묻는 메시지가 나타나면 **Don't Save**를 클릭
6. 다시 World를 로드하려면 **LOAD** 버튼을 클릭
7. `hello_world.py` 을 열고 **"Ctrl+S"**를 눌러 **Hot-Reload** 기능을 사용



재시작되어 Workspace 내 메뉴가 사라지는 것을 확인할 수 있습니다.

8. **Hello World Example**를 다시 열고 **LOAD** 버튼을 클릭

Adding to the Scene

https://prod-files-secure.s3.us-west-2.amazonaws.com/5cf11f82-c56d-4cc4-b334-8b9699104c71/a4aff64b-8817-4741-9c6d-836f17fb5f34/Adding_to_the_Scene.mp4

1. **Python API**를 사용하여 화면에 Cube를 추가하기 위해 `hello_world.py` 을 열고 다음과 같이 코드를 수정하고 저장

```
from omni.isaac.examples.base_sample import BaseSample
import numpy as np
# Can be used to create a new cube or to point to an already existing cube in stage.
from omni.isaac.core.objects.cuboid import DynamicCuboid

class HelloWorld(BaseSample):
    def __init__(self) -> None:
        super().__init__()
        return

    def setup_scene(self):
        world = self.get_world()
        world.scene.add_default_ground_plane()
        fancy_cube = world.scene.add(
            DynamicCuboid(
                prim_path="/World/random_cube", # The prim path of the cube in the USD stage
                name="fancy_cube", # The unique name used to retrieve the object from the scene later on
                position=np.array([0, 0, 1.0]), # Using the current stage units which is in meters by default.
                scale=np.array([0.5015, 0.5015, 0.5015]), # most arguments accept mainly numpy arrays.
                color=np.array([0, 0, 1.0]), # RGB channels, going from 0-1
            ))
        return
```

2. **"Ctrl+S"**를 눌러 **Hot-Reload** 기능을 사용
3. 메뉴를 다시 열기
4. **File > New From Stage Template > Empty**를 클릭하고 **LOAD** 버튼을 클릭



setup_scene에서 무엇이든 변경하는 경우, 해당 작업을 반드시 실행해야 합니다. 그렇지 않은 경우, **LOAD** 버튼만 클릭합니다.

5. **Play** 버튼을 눌러 시뮬레이션을 시작하고 Cube가 떨어지는 모습을 확인

Adding a New Example in the Menu

https://prod-files-secure.s3.us-west-2.amazonaws.com/5cf11f82-c56d-4cc4-b334-8b9699104c71/44ba397e-c6c8-416b-b6bf-4d5f2995b729/Adding_a_New_Example_in_the_Menu.mp4

1. **Isaac Examples** 메뉴 아래에 새 Example를 만들기 위해 `hello_world.py` 와 `hello_world_extension.py` 파일을 복사
2. `exts/omni.isaac.examples/omni/isaac/examples/user_examples` 폴더로 붙여넣기
3. `exts/omni.isaac.examples/omni/isaac/examples/user_examples/__init__.py` 에 다음 코드를 추가하고 저장

```
from omni.isaac.examples.user_examples.hello_world import HelloWorld
```

```
from omni.isaac.examples.user_examples.hello_world_extension import HelloWorldExtension
```

4. `exts/omni.isaac.examples/omni/isaac/examples/user_examples/hello_world_extension.py` 을 다음과 같이 코드를 수정하고 저장

```
import os
from omni.isaac.examples.base_sample import BaseSampleExtension
from omni.isaac.examples.user_examples import HelloWorld

class HelloWorldExtension(BaseSampleExtension):
    def on_startup(self, ext_id: str):
        super().on_startup(ext_id)
        super().start_extension(
            menu_name="",
            submenu_name="",
            name="Awesome Example",
            title="My Awesome Example",
            doc_link="https://docs.omniverse.nvidia.com/app_isaacsim/app_isaacsim/tutorial_core_hello_world.html",
            overview="This Example introduces the user on how to do cool stuff with Isaac Sim through scripting in asynchronous mode.",
            file_path=os.path.abspath(__file__),
            sample=HelloWorld(),
        )
        return
```

5. Isaac Examples 메뉴 아래 **Awesome Example**이 추가된 것을 확인하고 **LOAD** 버튼을 클릭하여 World를 로드

Converting the Example to a Standalone Application

https://prod-files-secure.s3.us-west-2.amazonaws.com/5cf11f82-c56d-4cc4-b334-8b9699104c71/a59b2168-9f85-4dbb-ad76-246f301cf785/Converting_the_Example_to_a_Standalone_Application.mkv

1. `my_application.py` 파일을 생성해 다음 코드를 추가하고 `exts/omni.isaac.examples/omni/isaac/examples/user_examples` 에 저장

```
#launch Isaac Sim before any other imports
#default first two lines in any standalone application
from isaacsim import SimulationApp
simulation_app = SimulationApp({"headless": False}) # we can also run as headless.

from omni.isaac.core import World
from omni.isaac.core.objects.cuboid import DynamicCuboid
import numpy as np

world = World()
world.scene.add_default_ground_plane()
fancy_cube = world.scene.add(
    DynamicCuboid(
        prim_path="/World/random_cube",
        name="fancy_cube",
        position=np.array([0, 0, 1.0]),
        scale=np.array([0.5015, 0.5015, 0.5015]),
        color=np.array([0, 0, 1.0]),
    ))
# Resetting the world needs to be called before querying anything related to an articulation specifically.
# Its recommended to always do a reset after adding your assets, for physics handles to
```

```

be propagated properly
world.reset()
for i in range(500):
    position, orientation = fancy_cube.get_world_pose()
    linear_velocity = fancy_cube.get_linear_velocity()
    # will be shown on terminal
    print("Cube position is : " + str(position))
    print("Cube's orientation is : " + str(orientation))
    print("Cube's linear velocity is : " + str(linear_velocity))
    # we have control over stepping physics and rendering in this workflow
    # things run in sync
    world.step(render=True) # execute one physics step and one rendering step

simulation_app.close() # close Isaac Sim

```

2. **cmd.exe**를 실행하여 다음 Command line을 차례로 입력

```

cd C:\Users\{user}\AppData\Local\ov\pkg\isaac-sim-4.0.0
doskey PYTHON_PATH=C:\Users\{user}\AppData\Local\ov\pkg\isaac-sim-4.0.0\python.bat $*
PYTHON_PATH ./exts/omni.isaac.examples/omni/isaac/examples/user_examples/my_application.py

```

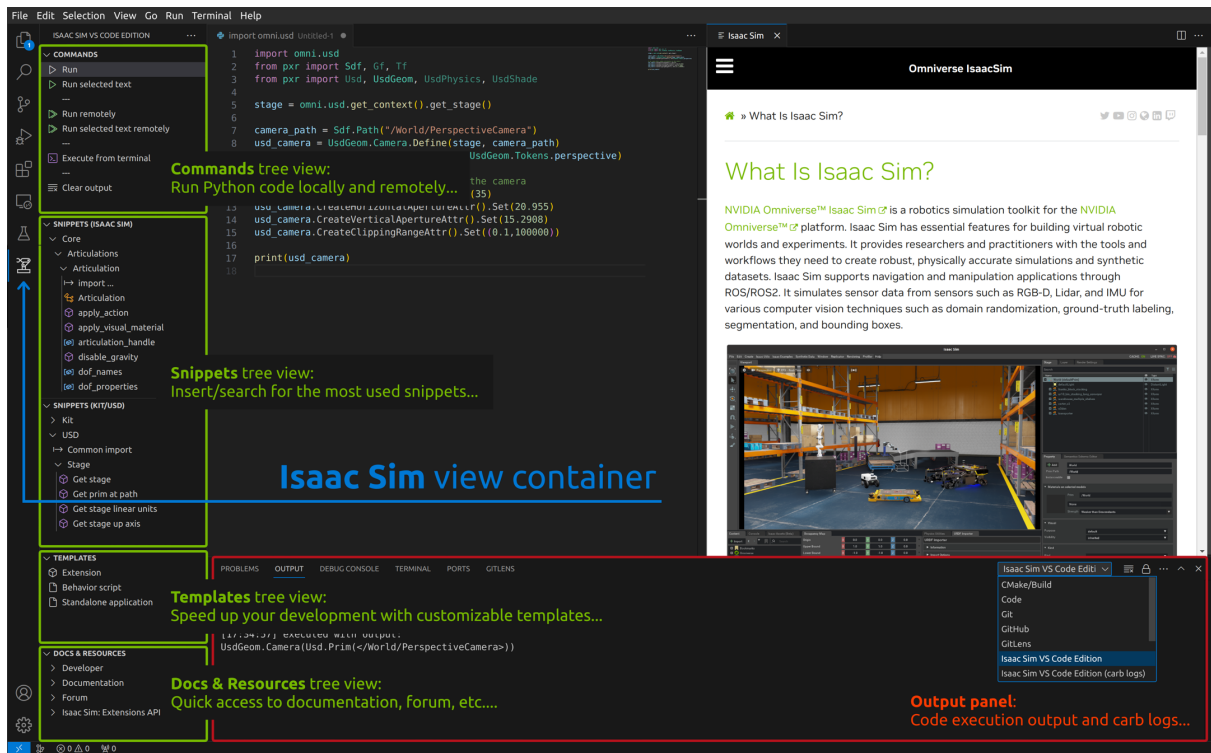
실습 - Running Python Code from External Editors

[REF] https://docs.omniverse.nvidia.com/isaacsim/latest/advanced_tutorials/tutorial_advanced_code_editors.html#

외부 편집기를 사용하여 Isaac Sim 애플리케이션 내에서 (Omniverse Script Editor를 사용할 때와 마찬가지로) Python 스크립트를 편집하고 실행할 수 있습니다.

VS Code

https://prod-files-secure.s3.us-west-2.amazonaws.com/5cf11f82-c56d-4cc4-b334-8b9699104c71/4f0cd174-c070-4e0a-8aa0-c95c949294f1/VS_Code.mkv



1. 새 파이썬 파일을 생성하여 원하는 위치에 저장

```
from pxr import Usd, UsdGeom
import omni

stage = omni.usd.get_context().get_stage()
xformPrim = UsdGeom.Xform.Define(stage, '/hello')
spherePrim = UsdGeom.Sphere.Define(stage, '/hello/world')
```

2. **Extension Manager**를 사용하여 `omni.isaac.vscode`을 검색해 확장 프로그램을 활성화
3. 확장 프로그램을 활성화한 후, 상단 메뉴 표시줄 내 **Window > VS Code**를 클릭하여 VS Code application에서 Isaac Sim 폴더 열기
4. VS Code App에서 Activity Bar (Isaac Sim 로고가 있는 막대)에 있는 **Isaac Sim VS Code Edition**을 클릭

<https://marketplace.visualstudio.com/items?itemName=NVIDIA.isaacsim-vscode-edition>
Isaac Sim VS Code Edition Extension을 VS Code에서 설치

5. 새 파이썬 파일 열고 **RUN**을 클릭

실습 - Attaching the Debugger to a Running App

[REF]

https://docs.omniverse.nvidia.com/isaacsim/latest/advanced_tutorials/tutorial_advanced_python_debugging.html#attach-the-debugger-to-a-running-app

Standalone Python Scripts



VS Code의 Open Folder로 가능

▼ In Linux

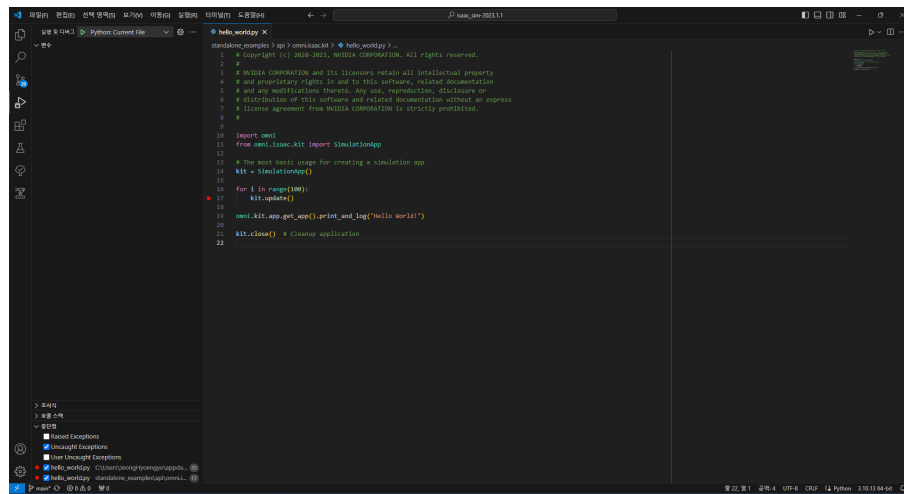
💡 Standalone Python Scripts 디버깅은 Linux에서만 지원합니다.

1. Isaac Sim App Selector에서 **Open in Terminal** 버튼을 클릭하고 다음 명령을 실행

code .

💡 새로운 VSCode 창이 시작되고 현재 폴더가 열리며 VSCode를 시작하고 폴더를 여는 것도 가능하다.

2. `standalone_examples/api/omni.isaac.kit/hello_world.py` 를 열고 `kit.update()` 에 Breakpoint를 설정
3. 왼쪽 도구 모음에서 **실행 및 디버그** 아이콘을 클릭하고 구성 메뉴에서 **Current File**이 선택되어 있는지 확인



4. **Start Debugging** 또는 **F5**를 클릭하여 Debugger를 시작
5. 현재 Debugging session을 중지하고 `.vscode/launch.json` 파일의 **"args"** 필드를 통해 기본 nucleus 서버를 변경

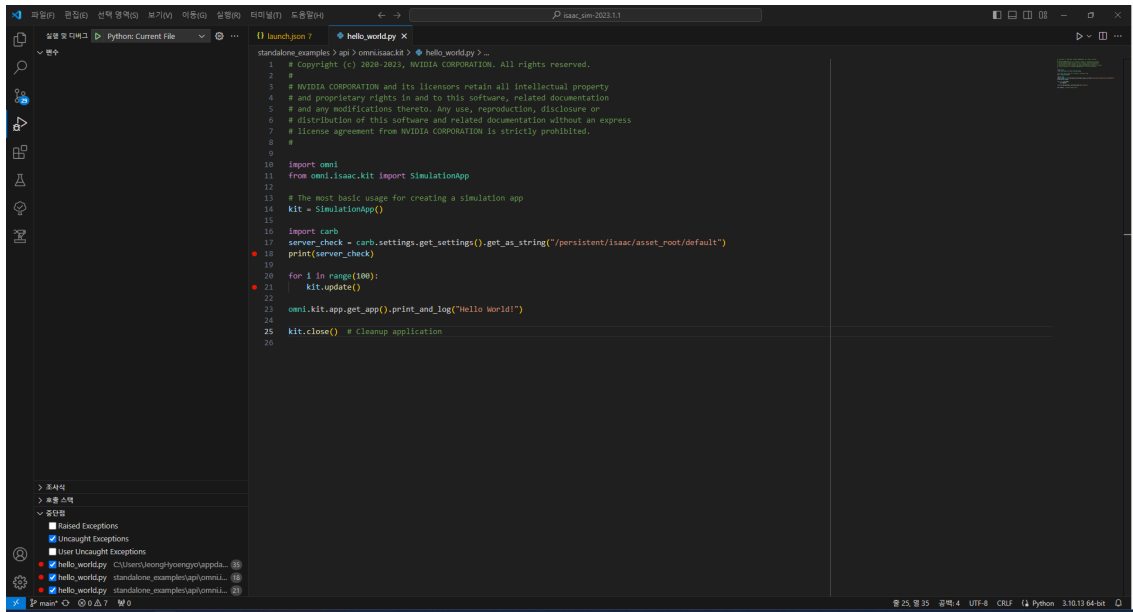
```
{
  "name": "Python: Current File",
  "type": "python",
  "request": "launch",
  "program": "${file}",
  "console": "integratedTerminal",
  "env": {
    "RESOURCE_NAME": "IsaacSim",
    "CARB_APP_PATH": "${workspaceFolder}/kit",
    "ISAAC_PATH": "${workspaceFolder}",
    "EXP_PATH": "${workspaceFolder}/apps",
    "PYTHONPATH": "${workspaceFolder}/site"
  },
  "python": "${workspaceFolder}\\kit\\python\\python.exe",
  "envFile": "${workspaceFolder}\\..\\.vscode\\.standalone_examples.env",
  "preLaunchTask": "setup_python_env",
  "args": ["--/persistent/isaac/asset_root/default=\"omniverse://my_server\""]
}
```

6. `hello_world.py` 을 열고 다음과 같이 코드를 수정

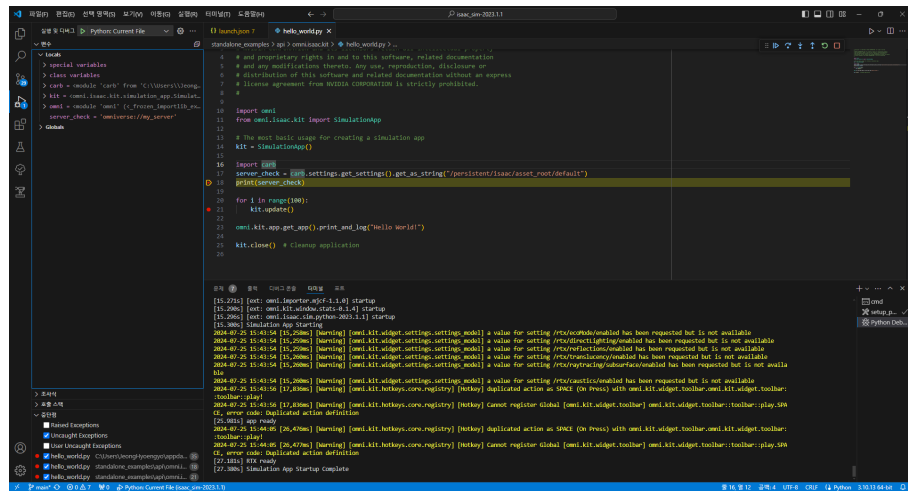
```
# The most basic usage for creating a simulation app
kit = SimulationApp()
```

```
import carb
server_check = carb.settings.get_settings().get_as_string("/persistent/isaac/asset_root/default")
print(server_check)
for i in range(100):
    kit.update()
kit.close() # Cleanup application
```

7. `print(server_check)` 에 Breakpoint를 설정



8. `launch.json` 을 수정하고 저장한 후, **F5**를 눌러 Debugger를 시작
9. 설정된 서버가 포함되어 있는지 확인



Attaching the Debugger to a Running App

https://prod-files-secure.s3.us-west-2.amazonaws.com/5cf11f82-c56d-4cc4-b334-8b9699104c71/b47860f6-c2c8-46a2-8a21-58967c214aa8/Attaching_the_Debugger_to_a_Running_App.mp4

1. Isaac Sim을 실행하고 상단 메뉴에서 **Window > Extensions**를 선택

2. VSCode를 검색하고 `omni.kit.debug.vscod` 확장 프로그램 **Enable** 버튼을 클릭
3. 기본적으로 상태는 빨간색 텍스트로 "**VS Code Debugger Unattached**"로 표시
4. VSCode를 실행하고 왼쪽 도구 모음에서 **실행 및 디버깅** 아이콘을 선택
5. 구성 메뉴에서 **Python: Attach (windows-x86_64/linux-x86_64)**를 선택하고 **녹색 화살표**를 클릭하여 디버깅을 시작
6. Isaac Sim의 상태가 파란색 텍스트로 "**VS Code Debugger Attached**"로 변경되는 것을 확인



만약 파란색 텍스트로 변경되지 않을 경우, `launch.json` 내 구성이 다음과 동일한지 확인해야 합니다.

```
{
  "name": "Python: Attach (windows-x86_64/linux-x86_64)",
  "type": "python",
  "request": "attach",
  "port": 3000,
  "host": "127.0.0.1"
},
```

7. 이제 VSCode에서 Python 파일로 돌아가 Breakpoint 추가하여 디버깅이 가능